PromptABI: Static Verification for the Discrete Interface Layer of LLM Systems

PromptABI Contributors

Abstract

Modern LLM systems fail in places that are not neural: chat templates splice untrusted text into role delimiters, tokenizer metadata changes the strings that grammars actually see, stop sequences fire inside tool arguments, provider adapter chains drop envelope fields, and framework truncation silently drops the evidence a prompt was meant to preserve. These failures are discrete, versioned, and usually present before a single token is generated, yet they are typically discovered through inference-time tests or production incidents.

PromptABI treats this boundary as an application binary interface for prompts. It gives templates, tokenizers, stop policies, tool schemas, provider envelopes, retrieval budgets, training manifests, and evaluation harnesses a common typed semantics. It then runs bounded symbolic execution, finite-language products, differential replay, and SMT-backed finite contracts to prove one of four useful facts: the interface is safe in the supported fragment, a concrete counterexample exists, the checker must abstain, or a versioned artifact has drifted from a previously verified contract.

The result is a CPU-only verifier that can be run in CI, before fine-tuning, before publishing benchmark scores, or before migrating providers. Its evidence is not a checklist of repository files: diagnostics carry executable witnesses, source spans, token paths, parser states, solver models, replay hashes, and privacy-preserving fingerprints. The reported bug-finding evaluation uses public bug replays that traverse exact pinned production-code excerpts.

1 Introduction

The software around an LLM has become a compiler pipeline. A user message is rendered by a chat template, transformed by tokenizer normalization and special token rules, packed into a context window, constrained by a grammar or JSON schema, streamed through stop policies, decoded by a provider adapter, and possibly reused as training or evaluation data. The model may be probabilistic, but this surrounding pipeline is largely deterministic. When it is wrong, the failure is often predictable without running the model.

PromptABI is built on that observation. It verifies the discrete interface layer of LLM systems before inference. The name is intentional: like an ABI for native code, the prompt interface defines what representations cross subsystem boundaries and which invariants downstream components assume. If a prompt template says that user content is data, but renders raw role delimiters inside that content, the ABI is broken. If a tool parser assumes that `</tool_call>` is structural, but a valid string argument may contain that sequence, the ABI is broken. If an evaluation harness scores a model after dropping the system prompt under a framework's packing policy, the ABI is broken.

This paper presents PromptABI as a programming-languages tool for LLM applications. The focus is not model quality, prompt style, or policy moderation. It is the statically checkable semantics of the artifacts that connect a model to the rest of a program.

Contributions. PromptABI contributes:

1. a typed interface model for prompt-system artifacts that are otherwise checked independently or only by example;
2. bounded semantics for chat-template rendering, tokenization, stop reachability, structured outputs, provider migration, context budgets, training masks, and evaluation harnesses;
3. proof-producing diagnostics that distinguish verified safety, concrete counterexamples, and honest abstention;
4. a deterministic CLI and embedding API designed for CI and offline use;

5. an evaluation suite whose reported bug-finding results come from public production code in the wild, not synthetic simplifications of those bugs.

2 Motivation

LLM applications are commonly tested end-to-end: run a model, inspect a sample, and patch the prompt when behavior looks wrong. That style misses interface bugs for three reasons.

The bug may be rare under sampling but inevitable in the interface. If a stop string is allowed inside a valid tool argument, generation may or may not produce it in a test run. The interface, however, admits the bad string regardless of the model distribution.

The responsible component is often not the model. Many incidents are caused by a tokenizer revision, chat-template update, provider adapter, streaming parser, framework truncation policy, or evaluation wrapper. These artifacts are versioned software inputs and can be checked like software inputs.

A passing run is not a proof. An inference test demonstrates one execution. A verifier can instead enumerate the bounded states of the interface, produce a minimal witness, and record why a claim is valid only in a supported fragment.

PromptABI addresses these gaps by moving checks left: the same artifacts that would be passed to inference are loaded, normalized, and checked before any weights or provider credentials are needed.

3 Core Abstraction

Definition 1 (Prompt interface artifact)**.** *A prompt interface artifact is a discrete, versioned object that influences the string or structured value seen by an LLM subsystem. Examples include chat templates, tokenizer configs, stop policies, JSON schemas, tool definitions, provider request/response envelopes, truncation policies, prompt segments, training manifests, and evaluation manifests.*

Definition 2 (Prompt ABI)**.** *A prompt ABI is the finite contract induced by a set of prompt interface artifacts: which roles may appear, which strings are data versus control, which tokens are reserved, which regions must survive packing, which grammars can be accepted, which stops may fire, and which structured fields downstream parsers expect.*

PromptABI compiles each artifact into small semantic views. A chat template becomes a bounded set of symbolic render paths and role regions. A tokenizer becomes an encode/normalize/decode relation over finite witnesses. A JSON schema becomes a grammar fragment and representative values. A provider adapter chain becomes a contract over fields, tool-call encodings, streaming chunks, stops, and structured-output modes, with per-hop preservation obligations checked against recorded provider snapshots. A training or evaluation manifest becomes a set of finite obligations about roles, masks, preserved context, answer schemas, and private fields.

The tool deliberately does not claim to model arbitrary Python, arbitrary Jinja, or arbitrary parser code. Its supported fragments are explicit. Outside them, PromptABI reports abstention instead of returning a false proof.

4 Formal Verification Core

PromptABI's checkers share a common pattern:

1. load typed artifacts with source spans, hashes, and version metadata;
2. compile them to bounded semantic objects;
3. run a product, symbolic execution, differential replay, or SMT query;
4. emit either a proof summary, a concrete witness, or an abstention.

4.1 Role Boundaries

The role-boundary checker symbolically executes supported chat-template fragments. It partitions each rendered path into structural regions such as system, user, assistant, tool, developer, and function. It then asks whether an attacker-controlled field can render a structural marker: role headers, special tokens, assistant prefixes, tool delimiters, markdown fences, or provider sentinels.

For example, a template that renders

```
'<|user|>\n' + message['content'] + '<|end|>\n'
```

without escaping permits user content to contain `<|system|>` or `<|end|>` as literal control text. PromptABI reports the input field, the marker, the rendered excerpt, the tokenized representation, and the exact offset of the forged boundary.

4.2 Stops and Structured Outputs

Stop policies are safe only relative to the language of outputs they interrupt. PromptABI builds bounded structured-output regions for JSON strings, tool arguments, XML-like tool calls, markdown fences, and provider tool-call envelopes. It then checks whether a configured stop sequence can occur before a valid structured output is complete.

The witness records the stop, the valid full output, the prefix returned after truncation, the parser state at the firing point, and the malformed or prematurely accepted structure a downstream parser would receive.

4.3 Tokenizers, Grammars, and Streaming Parsers

Tokenizer bugs often arise from mismatches between text-level constraints and token-level behavior. PromptABI can compare tokenizer revisions, detect drift in special-token IDs and normalization, and form bounded products between a tokenizer relation and a grammar. This exposes empty products, ambiguous token paths, decoded-text conflicts, whitespace aliases, Unicode normalization surprises, and added-token interactions.

Streaming parsers are first-class products as well. A provider may split a tool call, JSON object, or structured response across arbitrary chunks, while the application parser treats some states as protected data regions. PromptABI models the chunked parser as a bounded deterministic state machine and composes it with a substring monitor DFA. The command `promptabi streaming-parser -monitor '</tool_call>'` proves whether a literal can be observed while the parser is inside a JSON string, and returns the chunk index, event index, parser state, and excerpt when it can. This makes streaming stop/tool-parser bugs reproducible without calling a provider.

4.4 Budgets, Training, and Evaluation

Context windows are treated as finite packing problems. PromptABI combines declared segments, tokenizer-derived or declared token counts, output reserves, tool overhead, generation prompts, and framework truncation policies to prove whether required regions survive. The same machinery applies to RAG citations, training masks, supervised target roles, DPO/RLHF pair consistency, and evaluation harness prompts. A benchmark score should not be trusted if the grader rubric, answer schema, system prompt, or private answer key can be dropped or exposed by the interface.

4.5 Provider Migration

Provider migration is checked as a contract comparison, not as string matching. PromptABI compares request and response surfaces: tool-call IDs, JSON-object versus JSON-string arguments, parallel-call support, streaming chunk assembly, stop semantics, context limits, structured-output modes, and error envelopes. For adapter chains, it additionally proves that declared provider-envelope obligations survive every hop: the analyzer checks the ordered chain, the adapter's own preservation declaration, and the target provider's recorded surface. A LiteLLM-to-Azure OpenAI chain can therefore be proven equivalent for OpenAI-style tool envelopes, while a LiteLLM-to-Anthropic chain emits concrete losses for response fields, JSON-string arguments, streaming fragments, stops, and structured-output modes. The output is a set of migration diagnostics with concrete fields and suggested safe fixes.

5 What the Tool Can Do

Table 1 summarizes the main classes of checks. The table is organized by programmer need rather than by implementation module.

Need	PromptABI check	Evidence produced
Prevent prompt control injection	Role-boundary non-forgeability over bounded template paths	Forged marker, input field, rendered region, token evidence
Avoid premature parser truncation	Stop-overreachability over schemas, tools, and provider envelopes	Stop firing point, parser state, malformed prefix
Trust constrained decoding	Tokenizer × grammar emptiness and ambiguity checks	Token path, decoded text, grammar state, minimal string
Migrate providers safely	Provider, streaming, and adapter-chain envelope preservation	Lost field, changed encoding, broken hop, response-shape witness
Protect RAG, training, and eval invariants	Context-budget, mask, role, and private-field obligations	Survival proof, dropped-region witness, finite assignment
Maintain reproducible CI	Lockfiles, signed bundles, replayable solver certificates, sliced products	Artifact hash, diagnostic fingerprint, replay hash, minimal artifact cover
Block stale deploys	Kubernetes, Terraform, GitHub Environment, and release-system gates	Current bundle hash, reproducibility hash, gate manifest
Report live contract identity	Runtime attestation hooks for services exposing verified prompt, tokenizer, template, schema, and provider contracts	Service-scoped bundle hash, per-contract hashes, env/HTTP/Kubernetes/OTel hooks

Table 1: PromptABI capabilities stated as user-facing verification needs.

6 How It Proves Things

PromptABI uses proof techniques that match the artifact being checked. It does not use one universal solver for every problem.

Bounded symbolic execution. For supported chat-template fragments, PromptABI enumerates bounded control-flow paths and loop counts. Each path is a sequence of symbolic segments annotated with role ownership and source expressions. A finding is constructed only when a user-controlled expression is in a region whose control markers are known.

Finite-language products. Tokenizers, grammars, and parser fragments are compiled to finite relations for the witnesses under consideration. Products and projections answer emptiness, ambiguity, and reachability questions with concrete paths.

Counterexample slicing. For composed products that span templates, tokenizers, providers, policies, and parsers, PromptABI can compute the minimum-cost artifact slice that still covers the failing product facts. The slicer exhaustively searches the bounded product, records omitted artifacts and cut dependency edges, and emits a witness whose certificate distinguishes a necessary cross-artifact explanation from incidental configuration noise.

SMT-backed finite contracts. Static contracts that combine budgets, roles, stops, masks, and provider facts are lowered to finite SMT obligations. Solver models become counterexamples; unsatisfiable obligations become proof summaries. Reduced solver-replay files allow a result to be rerun without the original private artifacts.

Differential replay. Where a library is the specification, PromptABI runs small differential cases against real tokenizer/template/parser behavior. This catches divergence between the local abstraction and the implementation developers deploy.

Fail-closed abstention. If a construct is outside the supported fragment, a dependency is missing, a schema cannot be modeled, or a source-derived value cannot be recovered, PromptABI reports that fact. It does not silently turn an unsupported case into a safe result.

Theorem 1 (Bounded counterexample soundness)**.** *For a supported checker fragment, every unsafe PromptABI diagnostic contains a witness that is accepted by the artifact semantics used by that checker and violates the stated interface obligation.*

Proof sketch. Each checker constructs diagnostics from semantic objects, not from ad-hoc strings. The template checker builds a witness from an enumerated symbolic path, role region, and input expression. The stop checker substitutes the stop into a valid structured-output witness and records the parser state at truncation. The grammar/tokenizer check records a concrete token path through the product. The SMT layer records the satisfying assignment or unsat result for the lowered finite obligation. Tests replay these witnesses through the same checker semantics and, where applicable, real library adapters. □

Theorem 2 (Honest supported-fragment boundary)**.** *PromptABI's successful safety claims are scoped to explicit supported fragments; inputs outside those fragments produce abstentions or setup failures rather than passing diagnostics.*

Proof sketch. Artifact loaders and checkers validate dialects, parser support, dependency availability, schema fragments, and source-derived replay assumptions before claiming success. Unsupported constructs are recorded in the diagnostic model, and release tests require abstentions to remain visible. □

7 Implementation

PromptABI is a Python tool with a deterministic command-line interface and an embedding API. The implementation is organized around typed artifacts and typed diagnostics rather than around particular frameworks. The same verification result can be rendered as text, JSON, SARIF, GitHub annotations, HTML reports, signed bundles, or downstream integration payloads.

The CLI is intentionally CPU-only for the checks described here. That property is important: users can run PromptABI in pull requests, air-gapped builds, pre-commit hooks, release gates, fine-tuning preflight jobs, benchmark publication workflows, and provider migration reviews without model weights or provider credentials.

Privacy is part of the implementation contract. Diagnostics can expose exact structural witnesses for public fixtures, but private workflows can choose position-preserving redaction, hash-only evidence, or structural summaries while preserving stable fingerprints.

8 Deployed Workflows

PromptABI is intentionally presented as a tool, not only as a library of analyses. The implementation ships multiple workflows that exercise the same artifact semantics from different operational positions.

Pull-request verification. The core command is `promptabi verify`. It discovers or accepts a configuration file, loads typed artifacts, applies policy packs, runs selected checks, and emits deterministic text, JSON, SARIF, GitHub annotations, or static HTML. CI usage can require a lockfile, compare changed artifacts only, and upload SARIF without contacting an LLM provider.

Deployment gates. `promptabi deployment-gates` regenerates signed verification-bundle evidence and emits enforcement examples for Kubernetes admission policies, Terraform preconditions, GitHub protected environments, and internal release systems. Each example binds deployment metadata to the current bundle hash, reproducibility hash, signing-key identifier, and verification gate status.

Runtime attestation. `promptabi runtime-attestation` produces the complementary service-side payload: a process can report which verified prompt segments, tokenizers, chat templates, schemas, and provider contracts it is running. The output includes a deterministic manifest plus environment-file, HTTP JSON, Kubernetes annotation, and OpenTelemetry attribute hooks. Each hook carries the current signed bundle hash, reproducibility hash, signing-key identifier, service name, environment, and stable per-contract hashes, so inventory systems can answer "what PromptABI contract is live here?" without reading private prompt contents. Drift alarms are intentionally a separate policy layer; this workflow is the trustworthy self-reporting primitive.

Runtime drift alarms. `promptabi runtime-alarms` consumes that self-report and compares it with the newest lockfile, runtime policy pack, corpus baseline, and known-bad artifact manifest. This separates service inventory from enforcement: a running service can expose only non-content hashes and version metadata, while the platform decides whether the live contract has drifted from reviewed artifacts, violates approved signing-key or environment policy, fell behind corpus baselines, or matches a known-bad tokenizer, schema, provider, or prompt version.

Training preflight. `promptabi verify-training` builds or loads a training manifest and checks role layouts, tokenizer/template consistency, packing windows, loss masks, target spans, DPO/RLHF pair structure, data-loader adapters, and redaction constraints. It is designed to run before a costly fine-tuning job starts.

Prompt-pack governance. Reusable prompts are treated as packages with exported templates, roles, tool schemas, stops, supported model families, lockfiles, upgrade checks, signed provenance, public registry metadata, and offline mirrors. A consuming application can reuse package-level proofs and then add local context, RAG chunks, tools, or truncation rules. The newest composition layer also checks the exported chat-template markers against the configured tokenizer: a prompt pack can now prove that its structural markers have bounded tokenizer evidence, or fail with a concrete user-content witness when a downstream tokenizer admits a control token that the pack did not reserve.

Debugging and education. `promptabi explain` expands one diagnostic into artifact snippets, formal property, witness, likely symptom, and fix. `promptabi proofs` emits proof sketches and executable notebook artifacts for the core families: role-boundary non-forgeability, stop reachability, grammar emptiness, budget survival, and training-mask alignment.

Release and maintainer workflows. The repository includes compatibility audits, corpus verification, maintainer refresh, LTS planning, release readiness, theorem-to-test traceability, and benchmark leaderboards. These are not paper-only scripts; they are CLI commands that run against repository fixtures and examples.

9 Platform Integration Payloads

The stable integration API emits a single platform-facing report whose surfaces are specialized for downstream systems.

Surface	Payload	Intended consumer
CI provider	exit code, SARIF, annotations, runtime counts	GitHub Actions, Buildkite, Jenkins, internal merge gates
IDE extension	document-grouped spans, rule IDs, suggestions	VS Code, language-server bridges, editor plugins
Dataset platform	training/eval artifacts and private-field-safe diagnostics	fine-tuning and benchmark pipelines
Model registry	provenance, reproducibility hash, optional signed bundle	HF Hub, MLflow-style stores, internal registries
Internal platform	policy, guarantee mode, privacy, risk rollups	enterprise AI platforms and audit dashboards

Table 2: Downstream integration surfaces produced by the same verification run.

The key engineering decision is that these payloads are stable and non-content-first. They preserve hashes, fingerprints, offsets, modes, and artifact summaries; they do not require platform teams to parse human text or copy private prompt contents into central services.

10 Model Registry Gates

Model publication is a natural place to enforce prompt-interface evidence: registry promotion already records model IDs, aliases, stages, lineage, and artifacts. PromptABI adds a deterministic publication manifest for four common registry families:

1. Hugging Face Hub model repositories and model-card metadata;
2. internal registries with access-controlled attestation storage;
3. MLflow-style registered models and artifact directories;
4. generic artifact repositories such as OCI, S3, or Artifactory layouts.

The command

```
promptabi model-registry -config examples/model-registries/promptabi.json -targets examples/model-registri
```

builds a manifest containing the registry evidence surface, target-specific publication commands, signed bundle metadata, reproducibility hash, diagnostic fingerprints, and artifact summaries. A deployment system can require `ok: true` before changing a model alias to production.

This matters because model registries often track weights but not the prompt ABI that makes those weights usable. The same base model can be safe for one template and unsafe for another; one provider adapter may serialize tools differently from another; one tokenizer revision may invalidate an accepted structured-output grammar. PromptABI attaches evidence to the published model version without embedding private prompts or calling the model.

11 Deployment Gates

Deployment systems need a narrower contract than model registries: they need to know whether the artifact about to run is still the one PromptABI verified. The deployment-gate workflow builds that contract from the same signed bundle payload used for audit trails. The command

```
promptabi deployment-gates -config examples/minimal/promptabi.json -output-dir /tmp/gates
```

emits four executable gate examples plus a deterministic manifest. The Kubernetes example is a `ValidatingAdmissionPolicy` requiring deployment annotations for the current bundle and reproducibility hashes. The Terraform example uses `terraform_data` preconditions so plan/apply fails on stale PromptABI metadata. The GitHub Environments example regenerates the gate manifest inside a protected environment job and checks hashes before approval. The internal release-system example is a compact JSON policy for release orchestrators that compare declared deployment metadata with signed PromptABI evidence.

These are intentionally examples, not a new trust root. The trust root remains the local signing key and the reproducible bundle payload: config hash, artifact hashes, diagnostic fingerprints, lockfile state, and witness hashes. Deployment controllers can be simple because they only compare stable hashes and gate status. If the prompt template, tokenizer, schema, provider fixture, policy pack, or diagnostics drift, the bundle hash changes and the deployment gate fails closed.

12 Enterprise and Air-Gapped Operation

The repository contains enterprise features that are unusual for research verification prototypes. Policy packs can require checks, set severity thresholds, select supported fragments, tune solver timeouts, disable local summary files, and require approved provider fixtures. Access-control metadata records approved private artifact indexes, prompt packs, policy packs, and audit-bundle retention expectations.

Air-gapped usage is first-class. PromptABI can run with local fixtures, vendored wheels, vendored Z3 binaries or wheels, offline seed corpora, local prompt-pack mirrors, and no provider network calls. Signed verification bundles store enough reproducibility metadata for an audit trail: config hash, lockfile state, artifact hashes, diagnostic fingerprints, witness hashes, solver metadata, and optional non-sensitive excerpts.

The resulting operational posture is conservative. The verifier does not need model weights, does not need prompt text to leave the repository, and does not send telemetry. Teams that need strict privacy can emit hash-only witnesses and still keep stable diagnostic fingerprints for trend reports and suppressions.

13 SDKs and Stable Public API

PromptABI exposes two layers for downstream code. The Python package exports typed artifacts, diagnostics, sessions, reports, lockfiles, bundles, integration payloads, model-registry publication helpers, and renderers. Public API metadata is generated by `promptabi api-docs`; stable symbols carry a compatibility promise for the 1.x line, while provisional symbols remain importable for experimentation.

For non-Python consumers, the repository ships thin TypeScript, Go, and Rust clients. They intentionally do not reimplement PromptABI's analyses. Instead, they read stable payloads: diagnostic JSON, integration reports, lockfiles, and signed verification bundles. This is the right boundary for CI providers, IDEs, model registries, dataset platforms, and internal AI platforms: verification stays in one implementation, while consumers get stable schemas.

Client	Reads	Typical use
TypeScript	diagnostics, reports, lockfiles, bundles	web dashboards, VS Code bridges, registry frontends
Go	diagnostics, reports, lockfiles, bundles	CI runners and platform controllers
Rust	diagnostics, reports, lockfiles, bundles	high-assurance release gates and local tools

Table 3: Thin SDKs consume stable evidence instead of duplicating checks.

14 Examples as Executable Specifications

The repository examples are designed to be executable specifications. The minimal example demonstrates config loading, artifact hashing, schema and tool metadata, and deterministic diagnostics. Role-boundary examples show a buggy ChatML-style template and a fixed escaped version. Stop-policy examples exercise vLLM-style stop traces. Token-budget examples demonstrate must-survive segments, reserved output budgets, and framework truncation behavior.

End-to-end examples cover tool calling, JSON structured output, RAG truncation, provider migration, training alignment, and a GPU-free training quickstart. Each pair contains the application symptom, a buggy PromptABI config, and a fixed config. The examples are intentionally small enough for CI but grounded in real interfaces: provider JSON, tool schemas, tokenizer configs, training manifests, and application parsers.

The model-registry examples extend this pattern to deployment. They show how the same verification result can be attached to a Hugging Face Hub model, an internal registry record, an MLflow-style registered model, or a generic artifact repository. The example target manifest is executable through `promptabi model-registry`; tests assert that all four publication families are present and that the evidence is signed and reproducible.

15 Benchlines and Experiments

PromptABI's repository includes several benchlines, each serving a different claim.

Smoke performance. `benchmarks/benchmark_smoke.py` measures representative local checks: tokenizer analysis, grammar emptiness, stop checks, SMT-backed static contracts, budget checks, and corpus-wide verification. The goal is not GPU throughput; it is whether static verification is cheap enough to run before merge.

Real-bug benchmark. `promptabi corpus real-bug-benchmark` loads labeled real-world bug records and verifies source hashes, upstream references, and expected diagnostic families. The production-code replay benchmark additionally checks merged upstream bugfix PRs against pre-fix source excerpts and removed lines.

Adversarial corpus. `promptabi corpus adversarial` generates and replays cases for role delimiters, special tokens, Unicode normalization, JSON escaping, markdown fences, XML-ish tool tags, and provider envelopes. This is a stress suite for interface boundaries, not a model attack benchmark.

Evaluation corpus. `promptabi corpus evaluation` scores labeled cases from `fixtures/evaluation/labeled_corpus.json`. The output reports precision, recall-like agreement, abstention behavior, and witness quality on known safe/unsafe/unsupported cases.

Conformance suites. Grammar-backend, tokenizer-family, provider-fixture, and framework-truncation conformance suites compare PromptABI abstractions to real or fixture-backed library behavior. These suites defend against abstraction drift: if the checker no longer matches the deployed library fragment, it should fail or abstain.

Maintainer refresh.

Corpus release gate. `promptabi corpus verify` runs the maintained seed, structured-schema, provider-fixture, conformance, real-bug, evaluation, and SMT benchmark gates with release-blocking thresholds.

Maintainer refresh. `promptabi maintain refresh` updates corpus summaries, expected diagnostics, provider fixtures, release notes, and benchmark metadata. It makes evaluation maintenance reproducible rather than relying on informal manual updates.

Roadmap and evidence reports. `promptabi roadmap trends` converts dashboard history into structural-risk trend reports across repositories, teams, model families, provider migrations, and fine-tuning pipelines. `promptabi roadmap corpus-refresh` emits a read-only annual refresh procedure that can retire obsolete artifacts, add model families, update provider semantics, and preserve old benchmarks without destroying longitudinal evidence. `promptabi roadmap award` builds a concise award-submission brief from comparative studies, real-bug benchmarks, conference demos, and leaderboards, so claims remain evidence-bound rather than marketing copy. `promptabi roadmap teaching` turns executable examples and proof notebooks into university and internal-training labs. `promptabi roadmap research-agenda` publishes the next hundred compositional-verification, solver, prompt-pack, training-contract, and provider-standard conformance goals.

Workflow	Evidence source	Output
Historical trends	dashboard JSONL, corpus manifests	team/model/provider risk movement
Annual corpus refresh	seed, structured-schema, provider, real-bug corpora	read-only retirement and preservation plan
Award brief	comparative study, demos, leaderboard	concise claims, limitations, reproduction commands
Teaching materials	examples, proof notebooks, concept docs	five-module syllabus and executable labs
Research agenda	maintained roadmap generator	next 100 tracked research steps

Table 4: Roadmap reports convert repository evidence into maintainable external artifacts.

16 Diagnostic Quality

A PromptABI diagnostic is designed to answer five questions in one artifact: what invariant failed, where the source of the invariant lives, which semantic mode was used, what witness demonstrates the issue, and what low-blast-radius fix is plausible. The diagnostic model includes severity, source span, artifact provenance, check modes, fingerprint, suggestions, witness trace, and optional upstream issue links.

Witnesses are deliberately concrete. A role-boundary witness contains the attacker-controlled field, rendered excerpt, token evidence, and forged boundary. A stop witness contains the stop, valid full output, returned prefix, and parser state. A budget witness contains token counts, truncation decisions, and dropped required segments. An SMT witness contains a finite assignment or unsat evidence. This makes diagnostics actionable for engineers and replayable for maintainers.

PromptABI also includes witness validation, shrinking, and slicing. Counterexamples can be minimized by string length, token count, message count, schema size, or rule complexity; multi-artifact products can be sliced to the cheapest set of template/tokenizer/provider/policy artifacts that still explains the failed facts. The upstream bug-report generator then turns the minimized repro into a sanitized issue with artifact versions, reproduction commands, expected/actual behavior, and non-sensitive traces.

17 Versioning, Drift, and Lockfiles

LLM interface artifacts drift. Tokenizer IDs change, added tokens appear, normalizers shift, chat templates acquire generation prompts, provider envelopes change, stop policies move from framework defaults to provider overrides, and dataset preprocessing pipelines get rebuilt with new library versions. PromptABI treats these changes as contract events.

Lockfiles pin artifact hashes, upstream revisions, library versions, provider fixture versions, supported fragments, and diagnostic baselines. The `promptabi diff` workflow compares configurations and reports contract-breaking changes with witnesses. The `promptabi version-gate` workflow lets teams declare whether a change is patch-safe, minor-breaking, or major-breaking for a deployment.

Regression bisection then identifies the exact artifact update that introduced a failure. Maintainers can bisect tokenizer, template, schema, provider, or framework revisions without running a model. This makes PromptABI useful not only for initial verification but for long-lived production systems that upgrade their surrounding LLM software weekly.

18 Static Contract Language

Beyond built-in checks, PromptABI includes a static contract language for declaring allowed roles, required regions, forbidden delimiters, schema obligations, stop policies, solver-backed invariants, and cross-artifact assume/guarantee facts. The language can be formatted, linted, migrated, and composed across organization policies, prompt packs, app configs, RAG systems, fine-tuning datasets, and eval harnesses.

The compiler lowers rules to the same verification backends used elsewhere: automata products, SMT formulas, differential checks, and diagnostics with source spans. Composition is explicit about precedence. Assume/guarantee clauses make dependencies reviewable: an app rule can assume that a tokenizer preserves a control-token contract or that a schema preserves a tool-call shape, and the verifier discharges that assumption only when a loaded provider artifact or artifact kind declares the matching guarantee. Linting catches impossible rules, vacuous guarantees, unsupported fragments, unresolved assumptions, contradictory policies, and overly broad suppressions. Migration explains behavior changes when rule syntax or solver-backed semantics evolve.

This is important because production teams rarely want only built-in checks. They want local invariants: "this system message must survive," "these tool names are the only allowed tools," "user text must not enter assistant loss," or "this provider migration cannot change argument encoding." Static contracts make those policies versioned and reviewable.

19 Governance and Community Infrastructure

PromptABI includes contribution and governance machinery because verification tools fail when corpora, proofs, and supported fragments are informal. Governance docs specify checker acceptance criteria, proof standards, corpus licensing, security disclosures, and release-blocking regressions. Maintainer health checks cover triage labels, release checklists, corpus review procedures, rotation docs, and long-term project metrics.

Community workflows accept sanitized bug fixtures, minimized witnesses, prompt-pack metadata, provider fixtures, and training-manifest adapters. Plugin certification tests check that third-party loaders, renderers, providers, and solver encodings preserve diagnostic and privacy contracts. A marketplace-style index records plugin capabilities, supported fragments, privacy behavior, version compatibility, and verification status.

20 Why Static Interface Verification Is Distinct

PromptABI occupies a narrow but valuable point in the LLM tooling space. It does not compete with model evals, red-team prompts, jailbreak classifiers, or semantic safety policies. Those tools ask what a model might do. PromptABI asks whether the deterministic interface surrounding the model is internally consistent before the model acts.

The distinction creates practical advantages. The verifier is CPU-only, so it can run in every pull request. It is version-aware, so a dependency update can be gated. It is source-mapped, so a diagnostic points to the artifact that created the contract. It is privacy-preserving, so teams can share hashes and structural witnesses without sharing prompts. And it is honest about supported fragments, so unsupported cases become visible engineering work rather than silent false proofs.

21 Evaluation Design

PromptABI's evaluation separates verifier-maintenance tests from real-world claims. Unit tests, property tests, and small local fixtures are useful for maintaining the implementation, but they are not counted here as bug-finding evidence. The evaluation in this paper counts only public upstream code that was shipped in the wild and is pinned by repository, commit, path, license, and SHA-256 hash.

21.1 Research Questions

The evaluation asks four questions.

1. **Bug finding on real code:** when given exact production artifacts from public incidents, does PromptABI find the interface bug?
2. **Attribution:** does the diagnostic identify the production artifact and boundary responsible for the bug?
3. **Replayability:** can the result be rerun offline from pinned public source bytes and upstream bug records without model weights, provider calls, or private data?
4. **Cost:** is the check cheap enough for CI?

21.2 GitHub Bug Search and Corpus

We searched public GitHub issues and pull requests in llama.cpp, vLLM, and Hugging Face Transformers for recorded failures involving chat templates, role delimiters, tool parsers, stop strings, quote sentinels, streaming parser state, and structured-output boundaries. That search identified dozens of PromptABI-class records, but a record is admitted to the strict analyzer evaluation only if the issue or PR names an upstream production artifact and PromptABI can recover the relevant boundary from exact pinned source bytes. Six records meet that bar in the current corpus. Records that require hand-reduced payloads, live model sampling, or a verifier family not yet implemented are excluded from the strict analyzer count.

To answer the scale question separately, we also mined merged upstream bugfix PRs and admitted only cases where the PR base commit, production file, and bugfix patch agree: the exact source excerpt at the base commit must contain source line(s) removed by the upstream fix. This creates a larger source-and-patch replay benchmark of real production bugs. It is deliberately reported separately from the PromptABI-specific analyzer result: it measures whether the benchmark is operating on real code and real upstream bug records, not whether every bug belongs to PromptABI's current verifier families.

Table 5 lists the strict PromptABI analyzer corpus. Each row is exact code in the wild. The tool loads the pinned source excerpt, validates the source hash, extracts the template, parser delimiter, parser regex, or parser control-flow pattern from those bytes, and only then runs the checker. If the expected value cannot be recovered from the source, the replay fails.

21.3 Results

PromptABI agrees with all six GitHub-recorded bugs in the strict analyzer corpus. The Gemma4 quote case finds the exact production parser pattern described in vLLM issue #39069: `_ESCAPE_TOKEN` is replaced with a raw quote, then fallback parsing uses a regex fragment that stops at the first internal quote. The Gemma4 streaming case finds the source pattern from issue #38910 and PR #38909: buffered delta text is reconstructed into the parser's accumulated input. The llama.cpp TAG_WITH_TAGGED case finds the source branch described in issue #21771, where non-string tagged parameters enter `p.json()` inside the tag grammar. The Phi case yields 18 role-boundary witnesses after traversing 430 bytes of the exact upstream template. The Qwen3 Coder case yields 2 stop-overreach witnesses after extracting `</tool_call>`, `</function>`, and `</parameter>` from exact upstream parser source. The Qwen3 XML case finds the stale function-state pattern from issue #43713.

The larger replay benchmark contains 66 exact upstream production-code cases: the six strict analyzer cases plus 60 merged upstream vLLM bugfix PRs admitted by source-and-patch agreement. For those 60 scale cases, PromptABI validates the PR URL, merged state, base commit, production source path, source excerpt hash, and that the exact line(s) removed by the upstream bugfix are present in the pinned pre-fix source excerpt. The 60 scale cases cover 90 upstream-removed source lines. On a local CPU run, the complete replay command finishes in 0.75 seconds wall-clock time.

Production artifact	Public incident	Property checked	Result
vLLM `gemma4_utils.py` at commit `6a11d72`	Issue #39069: internal quotes truncate Gemma4 tool arguments	Does source replace the Gemma quote sentinel with `"` and then parse with a quote-stopping fallback regex?	Bug found: parser truncation pattern
vLLM `gemma4_tool_parser.py` before PR #38909	Issue #38910: streamed HTML tags duplicated inside tool arguments	Does streaming buffer output get stitched back into the accumulated parser input?	Bug found: buffer reparse pattern
llama.cpp `chat-auto-parser-generator.cpp` at commit `2016bf2`	Issue #21771: Qwen3 TAG_WITH_TAGGED array-of-object arguments poison streaming history	Does the non-string tagged-parameter branch route structured arguments through `p.json()` inside the tag grammar?	Bug found: JSON-parameter boundary pattern
llama.cpp `microsoft-Phi-3.5-mini-instruct.jinja` at commit `2016bf2`	PR #18462: Phi-style role delimiter handling	Can raw message content forge role/special-token boundaries?	Bug found: 18 role-boundary witnesses
vLLM `qwen3coder_tool_parser.py` at commit `6a11d72`	PR #40861: Qwen3 XML tool parser regressions	Can parser stop delimiters occur inside a valid tool argument boundary?	Bug found: 2 stop-overreach witnesses
vLLM `qwen3xml_tool_parser.py` at commit `6a11d72`	Issue #43713: multiple `<function=...>` blocks emit invalid JSON	Does function-open auto-close reuse stale function state without a fresh tool-call boundary?	Bug found: stale function-state pattern

Table 5: Strict PromptABI analyzer cases. Every case is replayed from exact pinned upstream production code; synthetic reductions are not counted.

21.4 Interpretation

The evaluation has two claims, kept separate. The strict claim is that for the six PromptABI analyzer cases, PromptABI traverses the same public code that shipped upstream, recovers the relevant boundary from that code, and produces concrete diagnostics before inference. The scale claim is that the benchmark can also replay more than 50 real production bugfixes without synthetic reductions: each scale case is tied to a merged upstream PR, a pre-fix production source file, an exact source hash, and source lines removed by the upstream fix. We do not count those 60 scale cases as additional PromptABI boundary bugs unless a PromptABI verifier family actually analyzes them.

22 Threat Model, Limitations, and Non-Goals

PromptABI is not a model-behavior verifier. It does not prove that an LLM will answer truthfully, refuse harmful requests, or choose a tool wisely. It proves properties of the discrete interface that surrounds the model.

PromptABI is also intentionally bounded. Chat-template loops, grammar products, schema recursion, SMT domains, and context-budget facts are finite. That makes diagnostics replayable and CI-friendly, but it means claims must be read with their bounds and supported-fragment metadata.

Finally, PromptABI is not a sandbox. It avoids provider calls and model execution for verification, and it can operate on redacted evidence, but process isolation and secret governance remain deployment responsibilities.

23 Related Systems

PromptABI sits between several families of tools. Prompt linters and jailbreak tests look for risky strings or behavioral failures, but they do not generally compile tokenizer/template/provider artifacts into a shared semantics. Schema validators and constrained decoders help one structured-output layer, but they usually do not account for tokenizer drift, stop policies, provider envelopes, or downstream parsers. Formal methods tools can prove finite obligations, but they are rarely packaged around the artifacts LLM engineers already maintain. PromptABI's contribution is to bring PL-style static and differential reasoning to those concrete artifacts.

Metric	Value
Exact upstream production-code cases	66
Strict PromptABI analyzer cases	6
Strict cases with expected bug found	6
Merged upstream bugfix replay cases	60
Bugfix replay cases with patch/source agreement	60
Abstentions in strict analyzer cases	0
Model or provider calls	0
Strict witnesses/pattern agreements	30
Upstream-removed source lines replayed	90
Wall-clock runtime	0.75s

Table 6: Current production-code evaluation results.

24 Conclusion

LLM systems need a verifier for the deterministic software boundary around the model. PromptABI supplies that verifier. It treats prompt templates, tokenizers, stops, schemas, tools, providers, budgets, training manifests, and evaluation harnesses as a prompt ABI; compiles them to bounded semantics; and emits replayable proofs, counterexamples, or abstentions before inference.

The resulting workflow is practical because it is CPU-only, version-aware, source-mapped, privacy-conscious, and integrated with CI. It is credible because its diagnostics carry executable witnesses and because its real-bug evaluation traverses exact production code that shipped in the wild.